

Level 2. The robot software stack

Level 1 gave you three boxes: sense, think, act. This level opens the "think" box and shows the seven or eight boxes that live inside almost every real robot. Then it shows how those boxes talk to each other (messages and topics), how they agree on where things are (frames) and how they agree on when things happened (time). By the end you can draw a full robot stack from memory and explain every arrow on it.

Rough time: two evenings. There is still no code, but there are links to the code you have already written in the tutorial, because several of these boxes are algorithms you know.

2.1 From three boxes to the standard stack

Start with the picture from level 1:

```
SENSE  ----->  THINK  ----->  ACT
```

Now think about a robot that has to drive from the kitchen to the front door. What does "think" actually have to do, in order?

1. Turn raw sensor bytes into numbers with units. (drivers)
2. Turn those numbers into "there is a chair 2 m ahead". (perception)
3. Work out where the robot itself is. (state estimation, also called localisation)
4. Remember what the world looks like. (mapping)
5. Decide a route to the door. (global planning)
6. Decide how to move in the next second without hitting the chair. (local planning)
7. Turn "move at 0.5 m/s, turn left a bit" into motor currents. (control)
8. Decide what the robot should be doing at all right now: driving, charging, waiting for a lift, stopped because a person is too close. (supervisor, also called the behaviour layer)

Every wheeled robot, arm, drone and self-driving car has these boxes. The names vary a little from company to company, but the jobs do not. Here is each one.

Drivers. A driver is the box that talks to one piece of hardware. It knows the cable, the protocol and the quirks of that one device, and nothing else. In: raw bytes from a USB (universal serial bus) or Ethernet port, or a request to send a command. Out: a clean message with units and a timestamp, for example "a scan of 720 range values in metres, taken at time t". There is one driver per device: lidar driver, camera driver, motor driver, IMU driver. (An IMU is an inertial measurement unit: a chip that measures acceleration and turn rate.) Drivers have no robotics in them. The trade-off they embody is: one tiny box per device because being able to swap a device without touching anything else matters more than saving a box.

Perception. Perception turns sensor data into facts about the world. In: images, lidar scans, point clouds. Out: "objects" with a class and a position ("person at 3.2 m, bearing 10 degrees"), or a cleaned-up scan with the floor removed, or a list of lane lines. Example algorithms: an object detector on images, line fitting and RANSAC (random sample consensus, a way of fitting a line while ignoring outliers) on lidar points (the closest-pair and line-fitting code in [tutorial/06-robotics.md](#), section b4, is a small perception box). Perception is usually the most expensive box in processor time, and the one most likely to be wrong.

State estimation, also called localisation. This box answers "where am I and how fast am I moving". In: wheel encoder counts, IMU readings, a GPS fix if outdoors (GPS is the global positioning system, a satellite position fix good to a few metres), and sometimes matches between the current lidar scan and the map. Out: the robot's pose (position plus heading) and velocity, with a measure of how sure it is. The classic

algorithm is the Kalman filter: the 1D and multi-dimensional versions you wrote in `tutorial/06-robotics.md`, section b1, are exactly this box for a robot moving in a line. The box runs fast, typically 50 to 200 times a second, because everything downstream needs a fresh pose.

Mapping. Mapping remembers what the world looks like. In: the current pose from state estimation and the current scan from perception. Out: a map. The most common map for a ground robot is an occupancy grid: a 2D grid of cells where each cell is "free", "occupied" or "unknown". That is the same `vector<vector<int>>` grid you did BFS on in `tutorial/03-grids-bfs-dfs.md`. Mapping and localisation depend on each other (you need a map to localise, you need a pose to map), and doing both at once is called SLAM, simultaneous localisation and mapping. Mapping can be slow, a few times a second, because the world changes slowly.

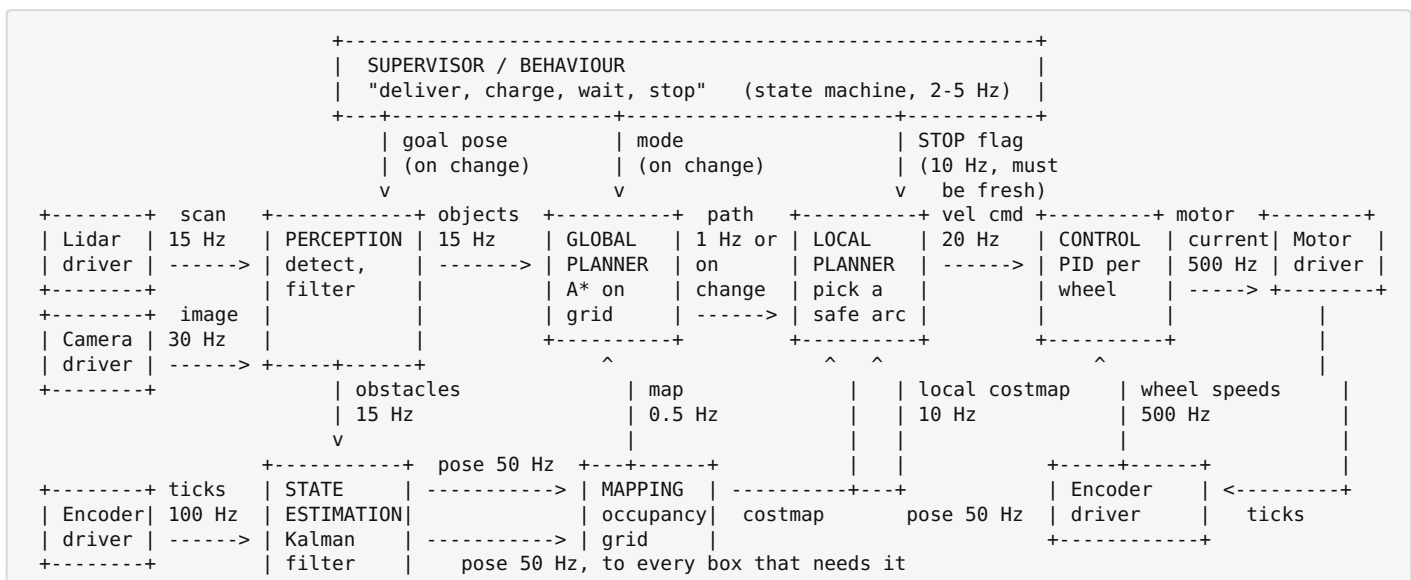
Global planning. Given the map, where I am, and where I want to go, find a route. In: the occupancy grid, start pose, goal pose. Out: a path, a list of (x, y) points from start to goal. Example algorithms: BFS on the grid when every step costs the same (`tutorial/03-grids-bfs-dfs.md`, section A2), Dijkstra when steps have different costs, for example rough ground (`tutorial/04-graphs-dijkstra.md`, section A5), A* when you want the same answer faster by heading towards the goal (`tutorial/03-grids-bfs-dfs.md`, Part C, the project you built). Global planning runs when the goal changes or when the map changes enough, roughly once a second or slower.

Local planning. The global path goes through the chair that appeared after the map was made. Local planning looks only a few metres ahead and a few seconds into the future and picks a safe motion that follows the global path as closely as it can. In: the global path, the current pose and velocity, and a small live map of the nearby obstacles (a local costmap). Out: a velocity command, "forward 0.4 m/s, turn 0.2 rad/s". Example algorithms: try a few hundred candidate arcs, simulate each for two seconds, score by "stays on path, stays away from obstacles, goes fast", pick the best. This runs 10 to 20 times a second.

Control. Control makes the wheels actually do what local planning asked. In: the velocity command, and the measured wheel speeds from the encoders. Out: motor voltages or currents. The classic algorithm is PID, proportional, integral, derivative: the controller with anti-windup you wrote in `tutorial/06-robotics.md`, section b2, is this box for one wheel. Control runs fastest of all, 100 to 1000 times a second, and often on a separate small board, because a slow control loop makes the robot wobble.

Supervisor, also called the behaviour layer. Somebody has to decide what the robot is doing right now. Is it delivering, charging, waiting for the lift, paused because a person stepped in front, or stopped because a sensor died? In: the state of every other box, the battery level, the task list, safety flags. Out: goals for the planner, mode switches, and the master "stop" flag. It is usually a state machine (a diagram of modes and the events that move between them) or a behaviour tree (a tree of "try this, if it fails try that"). It runs slowly, a few times a second, because decisions at this level are about seconds and minutes, not milliseconds.

Here is the whole thing on one page. Read it left to right. Sense is on the left, act is on the right, and the supervisor sits above everything because it talks to all of it.



```

+-----+ imu | |
| IMU | 200 Hz | |
| driver | -----> +-----+
+-----+

```

Two things to notice. First, the rates fall as you move right and up: encoders at 100 Hz, pose at 50 Hz, local planner at 20 Hz, global planner at 1 Hz, supervisor at a few hertz. Fast, small decisions at the bottom; slow, big decisions at the top. Second, one arrow is different from all the others: the STOP flag. It goes from the supervisor straight to control, skipping the planners. Section 2.5 explains why.

A note on what "Hz" means, because it will be on every arrow from now on: hertz is "times per second". 20 Hz is twenty times a second, one every 50 ms.

2.2 Messages, topics and the publish/subscribe idea

The noticeboard

Imagine an office with a noticeboard in the corridor. The lidar driver walks past fifteen times a second and pins a fresh note: "here is my latest scan". It does not know or care who reads it. Perception walks past and reads the latest scan note. So does mapping. So does a logging program that saves everything to disk. If tomorrow someone adds a fourth reader, the lidar driver does not change at all.

```

Lidar driver  ---> | NOTICEBOARD: "scan" | ---> Perception reads
pins a note      | latest note: 720 ranges, time t | ---> Mapping reads
15 times / s     | | | ---> Logger reads
                  +-----+
(writer does not know who reads)      (readers do not know who wrote)

```

This is called publish/subscribe. The writer *publishes*. A reader *subscribes*. The noticeboard is a *topic*. The note is a *message*. The whole point is that the writer and the readers are not joined together: you can replace, add or remove any of them without telling the others. That is the "component" idea from level 1 made real.

This idea is older than any robot framework and exists without ROS: MQTT (message queuing telemetry transport) in the Internet of Things, DDS (data distribution service) in aircraft, Kafka in web backends, all are noticeboards. In robotics the standard vocabulary comes from ROS 2, the Robot Operating System, which is not an operating system but a set of libraries and tools for building robot stacks. Its words are used in almost every UK robotics job advert, so learn them as plain ideas:

ROS 2 word	Plain idea	Example
node	one running program, one box on the diagram	the lidar driver
topic	a named noticeboard	/scan
message	one note, with a fixed layout of fields	720 floats, a frame name, a timestamp
publisher	a node that pins notes on a topic	lidar driver publishes to /scan
subscriber	a node that reads notes from a topic	perception subscribes to /scan

Drawn in ROS 2 words:

```

+-----+ /scan (LaserScan, 15 Hz) +-----+
| lidar_driver | -----> | perception |
| (node) | | | (node) |
+-----+ | +-----+
          | +-----+
          +-----> | mapping |
                    |

```

```
| (node) |
+-----+
```

The words in brackets on the arrow, LaserScan, are the message *type*: a fixed layout, "a header with a timestamp and frame name, then a start angle, an angle step, and an array of ranges". Every message on a topic has the same type. Two nodes agreeing on a message type is exactly "agreeing on an interface" from level 1.

The noticeboard has one important property: it is one-way and fire-and-forget. The lidar driver never finds out whether anyone read the scan. That is perfect for sensor data, where the next scan is coming in 66 ms anyway and nobody wants a stale one. It is wrong for some other jobs, which is why there are two more ways to talk.

Request/reply: asking a question and waiting

Sometimes a box needs an answer before it can continue. "Reset the odometry to zero." "What is the current map?" "Is the gripper open?" This is a *service*: one node asks, one node answers, and the asker waits for the answer. Like knocking on a colleague's door and asking a question, rather than pinning a note.

```
+-----+ request: "reset odometry" +-----+
| supervisor | -----> | state estimation |
|             | <----- |             |
+-----+ reply: "done, pose = 0,0,0" +-----+
              (once, when asked; asker waits)
```

The rule for services: use them for things that happen rarely, finish quickly, and where the asker truly needs the answer. Never for a stream of data, and never for anything that takes more than a fraction of a second, because the asker is stuck waiting.

Long-running tasks: ordering food and getting updates

"Navigate to the front door" takes thirty seconds. The supervisor does not want to sit frozen for thirty seconds. It wants to give the order, get "accepted", then get progress ("12 m to go", "8 m to go"), be able to cancel ("a person needs the lift, stop"), and finally get a result ("arrived" or "failed, door blocked"). This is an *action*. It is like ordering food in an app: you place the order, you see "preparing", "on its way", you can cancel before it leaves, and eventually it arrives.

```
+-----+ goal: "go to (12.0, 3.5)" +-----+
| supervisor | -----> | navigation |
|             | <----- | (global + local |
|             | <----- | planner) |
|             | <----- |             |
|             | <----- |             |
|             | <----- |             |
|             | <----- |             |
|             | <----- |             |
|             | <----- |             |
+-----+ (one goal, many feedback notes, one result; asker keeps working)
```

Under the surface an action is built out of the other two: a service to send the goal, a topic for the feedback, a service to fetch the result. You do not need to know that to use it, but it tells you the idea is not magic.

Which one to use

Pattern	Plain idea	Use when	Robot examples
topic	noticeboard, one-way, latest note wins	data that streams, many readers, stale notes are fine to drop	lidar scans, camera images, pose, velocity commands, battery level

Pattern	Plain idea	Use when	Robot examples
service	ask a question, wait for the answer	rare, quick, and the asker needs the reply before continuing	"reset odometry", "save the map", "open the gripper", "get parameter"
action	place an order, get progress, can cancel, get a result	takes seconds or longer, needs progress or cancel	"navigate to goal", "dock and charge", "pick the item in bin 3", "fly the inspection route"

The common mistakes: using a service for sensor data (the reader is stuck waiting and the writer cannot serve two readers), and using a topic for "navigate to goal" (there is no way to learn that it finished, and no way to cancel). If you can say "stream", it is a topic. If you can say "ask", it is a service. If you can say "order and wait for it to arrive", it is an action.

One trade-off to state clearly: a topic drops old notes because freshness matters more than completeness for sensor data. A service never drops a request because completeness matters more than freshness for commands like "save the map". ROS 2 lets you tune this per topic (a setting called quality of service, QoS: "keep the last 1 note" versus "keep all notes and retry"), and level 3 covers it.

2.3 Coordinate frames as a system concern

tutorial/06-robotics.md, section b3, taught you that a frame is a point of view: "the cup is at (1, 0)" means nothing until you say from where. That was one box's problem. In a full stack it becomes everyone's problem, because every box reports positions and no two boxes naturally use the same point of view.

The standard frames on a mobile robot, from the outside in:

```
map -----> odom -----> base_link -----> camera
                                           -----> lidar
                                           -----> imu
```

base_link. The robot's own body. Origin at the centre of the robot, x forward, y left, z up. Every sensor is bolted somewhere on the body, so every sensor frame is a fixed offset from base_link. "The lidar is 30 cm forward and 20 cm up." This offset never changes while the robot is running, so it is published once and remembered.

odom. Where the robot started when it was switched on, according to the wheels and the IMU only. The transform odom to base_link is what state estimation publishes at 50 Hz. It is smooth (it never jumps) but it drifts: after ten minutes it can be a metre off, exactly the vacuum-cleaner trade-off from level 1.

map. A fixed point in the building, the origin of the occupancy grid. The transform map to odom is the correction that localisation computes when it matches a lidar scan against the map: "the wheels think you are at (5.0, 2.0) but the walls say (5.4, 2.1), so shift odom by (0.4, 0.1)". This transform jumps when a correction arrives, and changes slowly, a few times a second.

Why two frames, odom and map, instead of one "where am I"? Because two different boxes want two different things. Control and local planning want a pose that is smooth, with no jumps, so they use odom. Global planning wants a pose that is correct in the long run, so it uses map. Splitting the drift into its own transform (map to odom) lets both be true at once. That is a design decision stated as a trade-off: two frames because smoothness for control matters more than having a single simple answer.

The TF tree as a system of boxes

The set of all frames and the transforms between them is called the TF tree (TF for transform). It is itself built with the noticeboard: every box that knows one transform publishes it, and a small library on the reading side stitches them into a chain.

```

+-----+ map -> odom (2 Hz) +-----+
| localisation | -----> | TF noticeboard |
+-----+ odom -> base_link (50 Hz) | (topic /tf) |
| state estimation | -----> | any box can ask: |
+-----+ base_link -> lidar, | "where was the camera |
| robot description | base_link -> camera, | in the map frame at |
| (published once, | base_link -> imu (static) | time t?" |
| from a URDF file) | -----> |
+-----+

```

(A URDF, unified robot description format, is a file that lists the robot's parts and where each is bolted. Its only job here is to be the source of the static transforms.)

The chain rule from the tutorial, $p_{\text{parent}} = R * p_{\text{child}} + t$ hop by hop, is exactly what the reading-side library does when perception says "person at (3.2, 0.5) in the camera frame" and the planner needs it in the map frame: camera to base_link (static), base_link to odom (from state estimation), odom to map (from localisation), three hops.

Why every sensor message must carry a frame and a timestamp

Look at the question in the diagram: "where was the camera at time t". Not "now". The robot is moving. A person seen in an image taken 80 ms ago must be placed using where the camera was 80 ms ago, not where it is now, or the person is drawn 8 cm from where they really are at 1 m/s (section 2.4 does this arithmetic). So the TF library keeps a short history of every transform, and every sensor message carries two things in its header:

- `frame_id`: which frame the numbers are in ("camera").
- `stamp`: when the measurement was taken.

A sensor box that publishes positions without a frame is publishing numbers with no meaning. A sensor box that publishes without a timestamp forces every reader to guess "probably now", and "probably now" is wrong by exactly the latency of the arrow. The rule for every driver you ever write: stamp the data with the time the sensor measured it, not the time your code received it, and name the frame.

2.4 Time

Every arrow on the stack diagram has a delay. Level 1 called it latency. Once a stack has eight boxes and thirty arrows, time stops being a detail and becomes something you design.

Why 80 ms matters

A camera at 30 Hz exposes for a few milliseconds, reads out, sends the image over USB, the driver unpacks it, perception runs a detector for 40 ms, and the result reaches the local planner. Adding up a typical chain: 33 ms to get the frame out of the camera, 40 ms to detect, 5 ms for the frame transform and fusion, and the local planner runs every 50 ms so on average you wait another 25 and at worst 50. Call it 80 to 130 ms from photon to plan.

At 1 m/s, a walking pace, 80 ms is 8 cm. The robot is 8 cm further along than the image shows. If the robot was turning at 1 rad/s (a brisk turn, about 57 degrees a second), 80 ms is 0.08 rad, 4.6 degrees, and an object 5 m ahead appears 40 cm to the side of where it really is. That is the width of a person. Ignore time and your robot plans around a ghost.

The fix is not "make everything faster", which is expensive and never enough. The fix is: stamp every message with when it was measured, and have each reader use the TF history to ask "where was the robot at that stamp". Then an 80 ms old image is placed correctly, it is just 80 ms old. Latency you know about is a nuisance. Latency you ignore is a bug.

Clocks

"When it was measured" needs a clock. There are usually three, and the mistake is to assume they agree:

- The **sensor's own clock**, if the sensor stamps its data itself (good lidars and cameras do). Accurate for intervals, but it starts from zero when the sensor powers up and drifts.
- The **computer's clock**. What `now()` returns in your C++ code. Also drifts: a cheap crystal at 20 parts per million loses about 72 ms an hour, 1.7 s a day.
- A **shared clock**, when there are several computers on the robot, or a fleet. They are kept together with a protocol such as PTP (precision time protocol) or NTP (network time protocol) so that "t = 1000.250" means the same instant on every machine.

The design rule is: pick one clock as the robot's time, and make every driver convert its sensor's stamps into that clock as the data comes in. A 72 ms per hour drift is nothing for a log file and everything for a control loop. Two boxes on two computers that disagree about "now" by 50 ms will place the same person in two different spots.

Simulated time

When you replay a recorded drive from a log file, or run the stack in a simulator, the clock must be the recording's clock, not the wall clock. Otherwise a message stamped at "t = 1000.250" arrives when the wall clock says three days later, every "how old is this" check fails, and the TF history throws everything away as stale. ROS 2 handles this with a flag (`use_sim_time`) that makes `now()` read from a `/clock` topic instead of the wall clock. The idea exists without ROS too: any stack that wants to be testable in replay needs a single "what time is it" function that can be switched from wall clock to log clock. Design it in from day one; retrofitting it means touching every box.

Pulling 2.3 and 2.4 together: the two things every message in a robot carries are a frame name and a timestamp. Everything else is payload.

2.5 Worked example: a delivery robot on a pavement

The job: carry a bag of shopping from a shop to a house 800 m away, on the pavement, at up to 1.5 m/s, without hitting anyone, and stop safely if anything goes wrong.

Sense. A 2D lidar for obstacles and localisation, a forward camera for recognising people, kerbs and crossings, wheel encoders, an IMU, a GPS receiver for the rough position outdoors, a bumper strip as the last line of defence, and a battery gauge.

Act. Two driven wheels, a lid lock, a small speaker and lights so pedestrians notice it.

Think. The standard stack from 2.1, plus a GPS input to localisation, plus a supervisor that also talks to a remote operator over a 4G mobile data link when it is stuck.

Here is the full stack. Every arrow has a message type and a rate.

```

+-----+
| SUPERVISOR (state machine, 5 Hz) |
| idle -> loading -> driving -> waiting_at_crossing -> driving -> arrived -> returning |
| any state -> paused (person < 1 m) -> stopped (fault) |
+-----+
| NavigateToGoal | mode | STOP (Bool, 10 Hz, must be | status (String, 1 Hz) |
| (action) | (String, | fresh; missing = stop) | to 4G link |
| v | on change) | v | v |

```

The interfaces, written down

8 / 11

Arrow	Message type	Rate	Roughly how much data
Bumper → Control	pressed (Bool)	on change	negligible

Two numbers stand out. The camera is 320 times more data than the lidar (27.6 MB/s against 86 kB/s). And the global map is 16 MB, which is why it is sent twice a second and not fifteen times: 15 Hz would be 240 MB/s of mostly unchanged grid.

Three trade-offs, stated properly

1. Lidar plus camera, or camera only? A camera-only robot is cheaper by a few hundred pounds and lighter. But a 2D lidar gives ranges to 0.03 m directly, in the dark and in rain, with 320 times less data to process, and it is what localisation matches against the map. Cameras give class ("that is a person, that is a bin") which a lidar cannot. The delivery robot keeps both: lidar because reliable range in all light matters more than saving the cost of one sensor, and camera because knowing "person" versus "post" changes how wide a berth to give. A camera-only design is valid for a robot that only runs in a well-lit warehouse and can afford a bigger computer.

2. How often to replan globally? At 1 Hz and 1.5 m/s, the robot moves 1.5 m between global plans. That sounds slow, but the global plan is only "which streets and which side of the road"; the local planner at 20 Hz handles everything within 7.5 cm of movement per update. Replanning globally at 10 Hz would cost ten A* runs a second over a 16 million cell grid for no benefit, because the map only changes twice a second anyway. So 1 Hz global, because compute matters more than reacting to a map that has not changed. If the map updated at 10 Hz (say, a busy crossing), the answer would change.

3. Where does the safety stop live? Three candidates. In the supervisor: it knows the most, but it runs at 5 Hz over the general-purpose computer, so between a person appearing and the stop command there can be 200 ms of supervisor period plus all the perception latency. In control: it runs at 500 Hz and is one arrow from the motors, but it knows nothing about people. In the motor driver hardware, wired to the bumper: 1 ms, knows nothing at all. The delivery robot uses all three, in layers. Perception-based slow-down ("person within 2 m, cap speed at 0.5 m/s") lives in the local planner. The STOP flag from the supervisor is checked by control at 500 Hz, and control treats a missing flag as a stop, so a crashed supervisor stops the robot rather than leaving it driving. The bumper goes straight to the motor driver and cuts current with no software in the way. The reasoning: the thing that must never fail should have the fewest boxes between it and the motors, because certainty of stopping matters more than cleverness about when to stop.

Check the arithmetic on that last one: at 1.5 m/s, a stop that lives in a 10 Hz loop adds up to 100 ms, which is 15 cm of travel. A stop in a 1 kHz motor board adds 1 ms, 0.15 cm. And the full stopping distance from 1.5 m/s with a 0.5 s reaction and 2 m/s² braking is 0.75 m of reaction plus 0.56 m of braking, 1.31 m. That is why the local planner starts slowing at 2 m, not at 1 m.

One more number for the report to the shop owner: a 24 V, 40 Ah battery holds 960 Wh; at 150 W average draw (motors, computer, lidar) that is 6.4 hours, about eight deliveries of 800 m each way plus waiting. Level 4 does battery and compute budgets properly.

2.6 How to describe a stack in two minutes

When you need to explain a robot stack to someone, on a whiteboard or in a design document, six sentences cover it. Fill in the brackets.

- The job.** "This robot [does X] in [environment], at up to [speed], and the thing it must never do is [hit a person / drop the load / leave the lane]."
- Sense.** "It senses with [sensors], of which [the lidar] is the one localisation depends on and [the camera] is the one perception depends on."

3. **Where am I.** "State estimation fuses [encoders, IMU, GPS, scan matching] into a pose at [50 Hz]; the map frame corrects the odom frame at [2 Hz]."
4. **Plan.** "A global planner runs [A*] on [an occupancy grid] at [1 Hz]; a local planner picks a safe velocity at [20 Hz] using a [6 m] costmap."
5. **Act.** "Control turns the velocity command into wheel currents at [500 Hz] with [PID]."
6. **Supervise and stop.** "A supervisor state machine at [5 Hz] issues goals as actions and a STOP flag; the STOP is checked in control, missing means stop, and the bumper cuts motor current in hardware, because [certainty matters more than cleverness there]."

Try it on the delivery robot without looking at 2.5. Then try it on your last Python project. If sentence 3 or 6 is empty, that is a gap in the project, not in the template.

2.7 Try it

Draw each of these as the full stack of 2.1, with every arrow labelled with message type and rate, and an interface table. Name three trade-offs for each, in the "X because Y matters more than Z" form. Do the arithmetic for any number you write down.

1. **A warehouse robot** that carries shelves to a packing station on a flat concrete floor with floor markers (small square barcodes) every metre. Questions to settle: is GPS present (no, indoors)? What replaces it for localisation? Does it need a camera at all, or is one downward marker reader enough? What rate does the fleet manager (a computer in the office that hands out jobs) talk to the supervisor at, and is that a topic, a service or an action?
2. **A drone inspecting a wind turbine blade.** The blade is 60 m long, the drone flies 5 m from it at 2 m/s, and a crack is 5 mm wide. Questions: how many pixels across is a crack, and does that set the camera or the distance? Where does the "return home if battery below 30 percent" decision live? The drone has no wheels, so what replaces the encoders in state estimation? How far does the drone move during a 100 ms image latency, and does that matter for a still photo of a crack?
3. **A robot arm picking items from a bin** with one camera above the bin. The arm has six joints, each with its own motor and encoder. Questions: what is the "map" here (hint: it is not an occupancy grid)? What replaces global and local planning (hint: a path is now a list of six joint angles over time)? What does the TF tree look like when the camera is fixed to the ceiling rather than the robot? Which of "detect the item", "plan a grasp", "execute the grasp" is a topic, a service and an action?

Level 5 works all three end to end, so write your answers down before you look.

2.8 In your own words

Every robot's "think" box opens into the same stack: drivers turn hardware into stamped messages; perception turns messages into facts about the world; state estimation says where the robot is; mapping remembers the world; a global planner finds a route on the map; a local planner finds a safe motion for the next second; control makes the motors do it; a supervisor decides what the robot is doing at all. Rates fall as you go up: control at hundreds of hertz, planners at tens, supervisor at a few.

Boxes talk through noticeboards: a publisher pins a message on a topic and any number of subscribers read it, without knowing each other. That is right for streams. For a quick question use a service. For a task that takes seconds and needs progress and cancel, use an action. The words are from ROS 2 but the ideas exist in every robot framework.

Every message carries a frame and a timestamp. The frame says which point of view the numbers are in; the TF tree of published transforms lets any box convert between map, odom, base_link and the sensors. The timestamp says when it was measured, which matters because the robot moved 8 cm during an 80 ms latency at walking pace. One robot clock, converted into by every driver, switchable to a log clock for replay. The safety stop has the fewest boxes between it and the motors, and a missing stop message means stop.

Check yourself

1. Name the eight boxes of the standard stack in order from sensor to motor, and give the typical rate of each. Which two run slowest and why?
2. Which tutorial file holds the code for the state estimation box? For the global planner? For the control box? Name the section in each.
3. A colleague proposes sending lidar scans to perception with a service call. Give two reasons that is the wrong pattern and say what the right one is.
4. "Navigate to goal" is an action. Name the four kinds of message that pass between the supervisor and the navigation box during one action, in order.
5. Why does a mobile robot have both an odom frame and a map frame? Which box publishes each transform, and which frame does the control box read its pose in?
6. A camera message arrives with no `stamp` field. What will every reader assume, and by how much will a person 5 m ahead be misplaced if the robot is turning at 1 rad/s and the true latency is 80 ms? Show the arithmetic.
7. Two computers on a robot disagree about the time by 50 ms. Describe one concrete symptom in the stack diagram of 2.5 and name the fix.
8. What is `use_sim_time` for, and what breaks in the TF library if you replay a log without it?
9. In the delivery robot, the global map is 16 MB and is published at 0.5 Hz. What would the data rate be at 15 Hz, and what is the trade-off sentence for keeping it at 0.5 Hz?
10. Give the three layers of the delivery robot's safety stop, from cleverest to most certain, and the rule for what control does if the STOP message stops arriving.